



Institute of
Information Technology

CYBERSECURITY LABORATORY

Lab Work 03

Linux Firewall with iptables

Packet Filtering, Stateful Inspection,
NAT, and Port Forwarding

Course: Intrusion Detection System and Intrusion Prevention System

All Slide Shows: <https://cygnussec.com/project/ids-ips>



What Is iptables

- The most popular firewall / **NAT** package running on Linux was ipchains. It had a number of limitations, the primary one being that it ran as a separate program and not as part of the kernel. The **Netfilter** organization decided to create a new product called **iptables** in order to rectify this shortcoming. As a result of this, **iptables** is considered a faster and more secure alternative. **iptables** has now become the default firewall package installed under RedHat and Fedora Linux.



What Is iptables (cont)

- The Linux kernel has the built-in ability to **filter** packets, allowing some of them into the system while stopping others. The 2.4 kernel's **netfilter** has three built-in tables or rules lists. They are as follows:
 - **filter**: The default table for handling network packets.
 - **nat**: Used to alter packets that create a new connection.
 - **mangle**: Used for specific types of packet alteration.



Packets Processing by the Firewall

Queue Type	Function	Chain	Chain Function
Filter	Packet filtering	FORWARD	Filters packets to servers accessible through another NIC on the firewall.
		INPUT	Filters packets destined for the firewall.
		OUTPUT	Filters packets originating from the firewall.
Mangle	TCP header modification	PREROUTING, POSTROUTING, OUTPUT, INPUT, FORWARD	Modifies TCP quality-of-service bits before routing; rarely used in SOHO environments.



Packets Processing by the Firewall

Queue Type	Function	Chain	Chain Function
NAT	Network Address Translation	PREROUTING	Translation before routing; used for destination NAT (DNAT).
		POSTROUTING	Translation after routing; used for source NAT (SNAT), including one-to-one or many-to-one NAT.
		OUTPUT	NAT for packets generated by the firewall; rarely used in SOHO environments.



What Is iptables

- The built-in chains for the **filter** table are as follows:
- **INPUT**-Applies to network packets that are targeted for the host.
- **OUTPUT**-Applies to locally-generated network packets.
- **FORWARD**-Applies to network packets routed through the host.
- The built-in chains for the **nat** table are as follows:



What Is iptables (cont)

- **PREROUTING**-Alters network packets when they arrive.
- **OUTPUT**-Alters locally-generated network packets before they are sent out.
- **POSTROUTING**-Alters network packets before they are sent out.



What Is iptables

- The built-in chains for the **mangle** table are as follows:
- **INPUT**-Alters network packets targeted for the host.
- **OUTPUT**-Alters locally-generated network packets before they are sent out.
- **FORWARD**-Alters network packets routed through the host.



What Is iptables (cont)

- **PREROUTING**-Alters incoming network packets before they are routed.
- **POSTROUTING**-Alters network packets before they are sent out.



Download -install - start the iptables

- Update Packages

```
sudo apt update
```

- Install **iptables** and persistence helper

```
sudo apt install -y iptables iptables-persistent
```

iptables-persistent helps keep your rules after reboot.

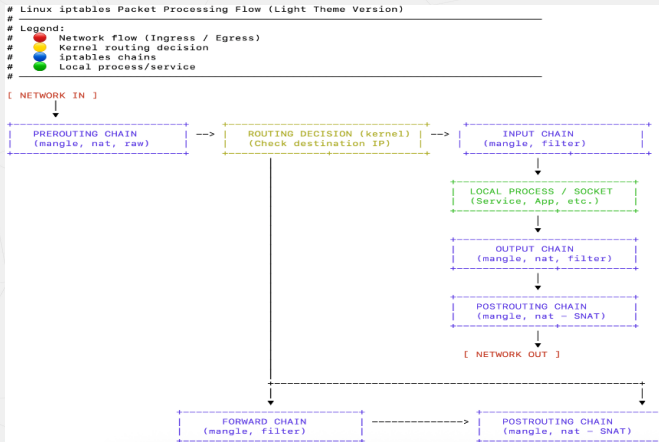
- Verify Installation

```
iptables --version
```

```
ip6tables -version
```



Packet Processing in iptables





Options Used in iptables Commands

- Rules that allow packets to be filtered by the kernel are put in place by running the **iptables** command. When using the iptables command, specify the following options:
 - Packet Type: Dictates what type of packets the command filters.
 - Packet Source/Destination: Dictates which packets the command filters based on the source or destination of the packet.



Options Used in iptables Commands (cont)

- Target: Dictates what action is taken on packets matching the above criteria.
- The options used with given **iptables** rule must be grouped logically, based on the purpose and conditions of the overall rule, in order for the rule to be valid.



Structure

- Many **iptables** commands have the following structure:

```
#iptables [-t <table-name>]
```

```
<command><chain-name> <parameter-1> <option-1>  
<parameter-n><option-n>
```

the default **filter** table to use with the command.



Structure (cont)

as appending or deleting the rule specified by the <chain-name> option.

define what will happen when a packet matches the rule

```
iptables -A INPUT -s 0/0 -i eth0 -d 192.168.1.1 -p TCP -j  
ACCEPT
```



Commands

- -A : Appends the **iptables** rule to the end of the specified chain.
- -F : Flushes the selected chain, which effectively deletes every rule in the chain. If no chain is specified, this command flushes every rule from every chain
- -L : Lists all of the rules in the chain specified after the



Commands (cont)

command. To list all rules in all chains in the default **filter** table, do not specify a chain or table. Otherwise, the following syntax should be used to list the rules in a specific chain in a particular table:

```
iptables -L <chain-name> -t <table-name>
```

- -N : Creates a new chain with a user-specified name.
- -P : Sets the default policy for a particular chain, so that when



Commands (cont)

packets traverse an entire chain without matching a rule, they will be sent on to a particular target, such as **ACCEPT** or **DROP**.

```
iptables -P INPUT DROP
```



The Most Commonly Used Targets

Target	Description	Common Options
ACCEPT	Stops further processing and hands the packet to the application or operating system.	N/A
DROP	Stops further processing; the packet is blocked.	N/A
REJECT	Blocks the packet and returns an error to the sender.	--reject-with; e.g. icmp-port-unreachable, icmp-net-unreachable, icmp-host-unreachable.



The Most Commonly Used Targets

Target	Description	Common Options
DNAT	Rewrites the destination IP address of the packet.	<code>--to-destination <ip-address></code>
SNAT	Rewrites the source IP address using an address selected by the user.	<code>--to-source <address></code> <code>[-<address>] [:<port>-<port>]</code>
MASQUERADE	Source NAT using the firewall interface address by default.	<code>--to-ports <port>[-<port>]</code> maps the original source port into a range.



General iptables Match Criteria

iptables Option	Description
<code>-t <table></code>	Selects a table; the default is <code>filter</code> . Built-in tables include <code>filter</code> , <code>nat</code> , and <code>mangle</code> .
<code>-j <target></code>	Jumps to the specified target when a packet matches.
<code>-p <protocol></code>	Matches <code>icmp</code> , <code>tcp</code> , <code>udp</code> , or <code>all</code> .
<code>-s / -d <address></code>	Matches source or destination IP address.
<code>-i / -o <interface></code>	Matches the input or output interface.



Parameters (1/7)

– **d** sets the destination hostname, IP address, or network that a packet must match. Supported network formats:

- N.N.N.N/M.M.M.M: IP range plus dotted-decimal netmask
- N.N.N.N/M: IP range plus prefix length



Parameters (2/7)

- `-s` sets the packet source using the same address syntax as `-d`.
- `-f` matches fragmented packets. Add `!` after this parameter to match only unfragmented packets.



Parameters (3/7)

-p selects the IP protocol for the rule.

- Common values: `icmp`, `tcp`, `udp`, and `all`
- Protocols listed in `/etc/protocols` may also be used
- If omitted, the default is `all`



Parameters (4/7)

-i selects the incoming interface, such as eth0 or ppp0.

- Filter table: valid with **INPUT** and **FORWARD**
- NAT/mangle tables: valid with **PREROUTING**



Parameters (5/7)

Special options supported by `-i`:

- `!` excludes the specified interface from the match
- `+` is a wildcard; for example, `-i eth+` matches Ethernet interfaces but excludes interfaces such as `ppp0`
- If no interface is specified, every interface is affected



Parameters (6/7)

–j tells iptables which target to use when a packet matches.

- Common targets: **ACCEPT**, **DROP**, **LOG**, and **REJECT**
- A rule may jump to a user-defined chain so additional rules can be applied
- Without a target, processing continues and the rule counter still increases



Parameters (7/7)

- **o** selects the outgoing interface.
- Filter table: valid with **OUTPUT** and **FORWARD**
- NAT/mangle tables: valid with **POSTROUTING**
- Supports the same special options as **-i**



Common TCP Match Criteria

Switch	Description
<code>-p tcp --sport <port></code>	TCP source port; accepts one value or a range start:end.
<code>-p tcp --dport <port></code>	TCP destination port; accepts one value or a range start:end.
<code>-p tcp --syn</code>	Matches a new TCP connection request.



Common UDP Match Criteria

Switch	Description
<code>-p udp --sport <port></code>	UDP source port; accepts one value or a range start:end.
<code>-p udp --dport <port></code>	UDP destination port; accepts one value or a range start:end.

```
iptables -A FORWARD -s 0/0 -i eth0 -d 192.168.1.58 -o eth1 -p tcp  
--sport 1024:65535 --dport 80 -j ACCEPT
```



TCP Protocol match options (-p tcp ...)

- `-dport` : Sets the destination port for the packet. To browse the names and aliases of network services and the port numbers they use, view the `/etc/services` file.
- To specify a specific range of port numbers, separate the two numbers with a colon (:), such as `-p tcp -dport 3000:3200`. The largest acceptable valid range is `0:65535`.
- Use an exclamation point character (!) after the `-`



TCP Protocol match options (-p tcp ...) (cont)

dportoption to tell **iptables** to match all packets which do not use that network service or port.



UDP Protocol match options (-p udp ...)

- `-sport` : Sets the source port of the packet using the same options as `-dport`.
- `-syn` : Applies to all TCP packets designed to initiate communication, commonly called SYN packets. Any packets that carry a data payload are not touched. Placing an exclamation point character (!) as a flag after the `-syn` option causes all non-SYN packets to be matched.
- `-dport` - Specifies the destination port of the



UDP Protocol match options (-p udp ...) (cont)

UDP packet.

- `-sport` - Specifies the source port of the UDP packet.



ICMP Protocol match options (-p icmp)

- These match options are available for the Internet Control Message Protocol (ICMP) (-p icmp):
- `--icmp-type<type>` : Sets the name or number of the ICMP type to match with the rule

```
iptables -A OUTPUT -p icmp --icmp-type  
echo-request -j ACCEPT
```

```
iptables -A INPUT -p icmp --icmp-type echo-reply -j  
ACCEPT
```



ICMP Protocol match options (-p icmp) (cont)

- In this example **iptables** is being configured to allow the firewall send ICMP echo-requests (pings) and in turn, accept the expected ICMP echo-replies.



Common Extended Match Criteria

Switch	Description
<code>-m multiport --port <p1,p2></code>	Matches multiple TCP/UDP source ports separated by commas.
<code>-m multiport --dport <p1,p2></code>	Matches multiple destination ports separated by commas.
<code>-m multiport --ports <p1,p2></code>	Matches multiple ports, assuming source and destination ports are the same.

```
iptables -A FORWARD -s 0/0 -i eth0 -d 192.168.1.58 -o eth1 -p tcp  
--sport 1024:65535 -m multiport --dport 80,443 -j ACCEPT
```



Common Extended Match Criteria

Switch	Connection States
<code>-m state --state <state></code>	ESTABLISHED: packets in an existing bidirectional connection. NEW: starts a new connection. RELATED: starts a secondary connection, such as FTP data or an ICMP error. INVALID: cannot be associated with a known flow.

```
iptables -A FORWARD -d 0/0 -o eth0 -s 192.168.1.58 -i eth1 -p tcp -m  
state --state ESTABLISHED -j ACCEPT
```



Modules with Additional Match Options

- To use a match option module, load the module by name using the `-m` option, such as `-m <module-name>` (replacing `<module-name>` with the name of the module).
- state module: The state module (`-m state --state ... match` a packet with the following connection states:



Modules with Additional Match Options (cont)

- **ESTABLISHED**-The packet is associated with other packets in an established connection.
- **INVALID**-The packet can't be matched to a known connection, and thus may be forged .
- **NEW**-The packet is trying to establish a new connection .



Modules with Additional Match Options (cont)

- **RELATED**-The packet is not part of an existing connection, but it's related, such as an ICMP error packet . These connection states can be used in combination with one another by separating them with commas, such as -m state --state **INVALID,NEW**.
- mac module-(-m mac ...) Enables hardware MACaddress matching



Modules with Additional Match Options (cont)

with the following option:

- `–mac-source` -Matches a MACaddress of the network interface card

that sent the packet. To exclude a MAC address from a rule, place an exclamation point (!) after the `–mac-source` match option.



Loading Kernel Modules Needed By iptables

- The **iptables** application requires you to load certain kernel

modules to activate some of its functions. Whenever any type of **NAT** is required, the iptable_**nat** module needs to be loaded. The ip_conntrack_ftp module needs to be added for FTP support and should always be loaded with the ip_conntrack module which tracks TCP connection states. As most scripts probably will keep track of connection



Loading Kernel Modules Needed By iptables (cont)

states, the `ip_conntrack` module will be needed in any case.

The

`ip_nat_ftp` module also needs to be loaded for FTP servers behind a **NAT** firewall.

```
# File: /etc/rc.local
```

```
# Module to track the state of connections
```

```
modprobe ip_conntrack
```



Loading Kernel Modules Needed By iptables (cont)

```
# Load the iptables active FTP module, requires  
ip_conntrack  
modprobe ip_conntrack_ftp  
# Load iptables NAT module when required  
modprobe iptable_nat  
# Module required for active an FTP server using NAT  
modprobe ip_nat_ftp
```



Example: Allowing DNS Access To Firewall

Allow outbound DNS queries from the FW and the replies too

Interface eth0 is the internet interface

Zone transfers use TCP and not UDP. Most home networks

websites using a single DNS server won't require TCP statements

```
iptables -A OUTPUT -p udp -o eth0
```



Example: Allowing DNS Access To Firewall (cont)

–dport 53 –sport 1024:65535 -j **ACCEPT**

`iptables -A INPUT -p udp -i eth0`

–sport 53 –dport 1024:65535 -j **ACCEPT**



Allowing WWW And SSH Access To Firewall

```
# Allow previously established connections
# - Interface eth0 is the internet interface
iptables -A OUTPUT -o eth0 -m state --state
ESTABLISHED,RELATED -j ACCEPT
# Allow port 80 (www) and 22 (SSH) connections to the
firewall
iptables -A INPUT -p tcp -i eth0 --dport 22
-sport 1024:65535 -m state --state NEW -j ACCEPT
```



Allowing WWW And SSH Access To Firewall (cont)

```
iptables -A INPUT -p tcp -i eth0 --dport 80  
-s sport 1024:65535 -m state --state NEW -j ACCEPT
```



Allowing Firewall To Access The Internet

Allow port 80 (www) and 443 (https) connections from the firewall

```
iptables -A OUTPUT -j ACCEPT -m state --state  
NEW,ESTABLISHED,RELATED -o eth0 -p tcp -m multiport  
-dport 80,443 -m multiport --sport 1024:65535
```

Allow previously established connections

- Interface eth0 is the internet interface



Allowing Firewall To Access The Internet (cont)

```
iptables -A INPUT -j ACCEPT -m state --state
```

ESTABLISHED,RELATED -i eth0 -p tcp If you want all TCP traffic originating from the firewall to be accepted, then remove the line: -m multiport --dport 80,443 -m multiport --sport 1024:65535



iptables Summary

- Flush the rules from a chain:

```
# iptables -F INPUT
```

```
# iptables -F OUTPUT
```

```
# iptables -F FORWARD
```

- Setting a Default Policy (-P or --policy)

```
# iptables -P INPUT DROP
```

```
# iptables -P OUTPUT DROP
```

```
# iptables -P FORWARD DROP
```



iptables Summary (cont)

- Creating Firewall Rules

```
# iptables --append CHAINselection-criteria --jump  
{DROP|ACCEPT|REJECT}
```

```
# iptables -A CHAINselection-criteria -j  
{DROP|ACCEPT|REJECT}
```

- Opening and Closing Specific Ports

```
# iptables -A INPUT -p tcp --dport 25 -j ACCEPT
```

```
# iptables -A OUTPUT -p tcp --sport 25 -j ACCEPT
```



iptables Summary

- Using Source and Destination IP Addresses

```
# iptables -A INPUT -s 172.24.0.0/16 -j DROP
```

```
# iptables -A OUTPUT -d 172.24.0.0/16 -j DROP
```

- Filtering by Interface

```
# iptables -A INPUT -s 192.168.9.0/24 -i eth0 -j DROP
```

```
# iptables -A FORWARD -s 192.168.9.0/24 -i eth0 -j DROP
```

```
# iptables -A FORWARD -s !192.168.9.0/24 -i eth1 -j DROP
```

```
# iptables -A OUTPUT -s !192.168.9.0/24 -i eth1 -j DROP
```



iptables Summary (cont)

- Performing Stateful Inspection

```
# iptables -A INPUT -m state -p tcp --dport 80 \  
-state NEW,ESTABLISHED,RELATED -j ACCEPT
```

```
# iptables -A OUTPUT -m state -p tcp --sport 80 \  
-state ESTABLISHED,RELATED -j ACCEPT
```



Masquerading(Many to One NAT)

- **NAT** allows a router to modify the contents of TCP/IP packets. In particular, **NAT** enables changes to the source and destination addresses of the packets. Masquerading is another word for what many call "many to one" **NAT**. In other words, traffic from all devices on one or more protected networks will appear as if it originated from a single IP address on the Internet side of the firewall

`iptables` requires the `iptables_nat` module to be



Masquerading(Many to One NAT) (cont)

loaded with the "modprobe" command for the masquerade feature to work. Masquerading also depends on the Linux operating system being configured to support routing between the internet and private network interfaces of the firewall. This is done by enabling "IP forwarding" or routing by giving the file `/proc/sys/net/ipv4/ip_forward` the value "1" as opposed to the default disabled value of "0".



Example of Masquerading

```
# Load the NAT module
modprobe iptable_nat

# Enable routing by modifying the ip_forward
# - Interface eth0 is the internet interface
# - Interface eth1 is the private network interface
echo 1 > /proc/sys/net/ipv4/ip_forward

iptables -A POSTROUTING -t nat -o eth0 -s \ 192.168.1.0/24
-d
```



Example of Masquerading (cont)

0/0 -j **MASQUERADE**

Prior to masquerading, the packets are routed via the filter

table's FORWARD chain.

Allowed outbound: New, established and related connections

Allowed inbound : Established and related connections

```
iptables -A FORWARD -t filter -i eth1 -m state \  
- - state NEW,ESTABLISHED,RELATED -j ACCEPT
```



Example of Masquerading (cont)

```
iptables -A FORWARD -t filter -i eth0 -m state \  
- state ESTABLISHED,RELATED -j ACCEPT
```



Forwarding Ports with iptables

- In many cases home users may get a single public IP address from their ISP. If their Linux firewall is their interface to the Internet and they want to host a website on one of the **NAT** protected home servers then they will have to use the "port forwarding" technique.
- Port forwarding is handled by the PREROUTING chain



Forwarding Ports with iptables (cont)

of the "nat" table. As in masquerading, the **iptables_nat** module will have to be loaded and routing enabled for port forwarding to work. Routing too will have to be allowed in **iptables** with the FORWARD chain, this would include all NEW inbound connections from the Internet matching the port forwarding port plus all future packets related to the ESTABLISHED connection in both directions.



Forwarding Ports with iptables

Port forwarding can be an extremely useful tool in certain situations, including the following:

- You move a server from one computer to another, but your DNS entries have not yet been properly updated. You can also use port forwarding to temporarily make such a change.
- You want a server to respond to multiple ports on a



Forwarding Ports with iptables (cont)

single computer. You can set up the system to forward one port to another on the same system. Some servers can listen directly to multiple ports, though, which is often a simpler approach.

- You want to run an externally accessible server within a **NAT**-protected network. You can set up the **NAT** router to forward traffic directed at one of its external ports to an internal system.



Example of port forwarding

```
# Load the NAT module and Enable routing
modprobe iptable_nat
echo 1 > /proc/sys/net/ipv4/ip_forward
external_int="eth0"
external_ip="203.132.168.1"
# Allow port forwarding for traffic destined to port 80
of
```



Example of port forwarding (cont)

the firewall's IP address to be forwarded to port 8080 on server 192.168.1.200. Interface eth0 is the internet interface
Interface eth1 is the private network interface

```
iptables -t nat -A PREROUTING -p tcp -i eth0 -d  
$external_ip -dport 80 --sport 1024:65535 -j DNAT --to  
192.168.1.200:8080
```



Example of port forwarding

After DNAT, the packets are routed via the filter's **FORWARD** chain.

Connections on port 80 to the target machine on the private network # must be allowed.

```
- iptables -A FORWARD -p tcp-ieth0 -o eth1 -d \
192.168.1.200 -dport 8080 -sport 1024:65535 -m \ state -
state NEW -j ACCEPT
- iptables -A FORWARD -t filter -i eth1 -m state \
```



Example of port forwarding (cont)

–state **NEW,ESTABLISHED,RELATED** -j **ACCEPT**

- iptables -A FORWARD -t filter -i eth0 -m state \

–state **ESTABLISHED,RELATED** -j **ACCEPT**



Institute of
Information Technology

Thank You For Listening!

Lab Work 03 – Linux Firewall with iptables